

segment. The value of the statistics do not have to be calculated from the raw data. Similarly, the instances of a weekly segment will be calculated from the instances of its child daily segment.

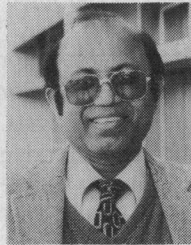
ACKNOWLEDGMENT

The author would like to thank Dr. A. Sporer and Dr. P. Mantey, of IBM Research Research Laboratory, for their constant encouragement and helpful suggestions during this research. Special thanks to Dr. A. Sporer for suggesting the investigation of testing by factorial design.

REFERENCES

- [1] "A LBL perspective on statistical database management," Lawrence Berkeley Lab., Univ. Calif., Tech. Rep., 1982.
- [2] A. Shoshani, "Statistical databases: Characteristics, problems and some solutions," in *Proc. VLDB Conf.*, Mexico City, Sept. 1982.
- [3] D. Bates, H. Boral, and D. J. DeWitt, "A framework for research in database management for statistical analysis or a primer on statistical database management problems for computer scientists," in *Proc. ACM SIGMOD Conf.*, 1982.
- [4] S. Y. W. Su, "SAM: A semantic association model for corporate and scientific/statistical databases," *J. Inform. Sci.*, vol. 3.
- [5] S. P. Ghosh, *Database Organization for Data Management*. New York: Academic, 1977.
- [6] G. E. P. Box, W. G. Hunter, and J. S. Hunter, *Statistics for Experimenters*. New York: Wiley, 1978.
- [7] D. Daniels *et al.*, "An introduction to distributed query compilation in R*," IBM, Res. Rep. RJ3497.
- [8] M. G. Kendall and A. Stuart, *The Advanced Theory of Statistics*. London, England: Charles Griffin, 1958.

- [9] L. H. Tippet, *Technological Applications of Statistics*. New York: Wiley, 1950.
- [10] W. A. Shewhart, *Economic Control of Quality of Manufactured Products*. Princeton, NJ: Van Nostrand, 1931.
- [11] W. A. Shewhart and G. Deming, *Statistical Method for the Viewpoint of Quality Control*. New York: Graduate Press.
- [12] D. B. Owen, *Handbook of Statistical Tables*. Reading, MA: Addison-Wesley, 1962.



Sakti P. Ghosh (M'69-SM'74) received the Ph.D. degree from the University of California, Berkeley.

He has been a Research Staff Member at IBM Research Division, San Jose Research Laboratory, San Jose, CA, since 1962. During this period he has made many original contributions to computer science and its applications to the business. His specialties are in the areas of data management, storage structures, performance and evaluation, information management systems, SQL, file structures, algebraic schemes, video disks, statistical database management systems, etc. He holds the position of Adjunct Full Professor at San Jose State University. He held the position of Assistant Manager for Education from 1979-1981 at IBM's Research Division. He holds two IBM Research Division Awards for research contributions and an IBM Invention Achievement Award for his patent contributions.

Dr. Ghosh is a member of many professional societies. He has published many technical papers in many journals and a textbook on database organization for data management, and has edited four more books. He was a member of IEEECS Education Activity Board and IEEECS Chapter Activities Board. He has been an IEEECS National Lecturer since 1977. He was the Program Co-chairman of the USA National Computer Conference in 1978. At present he is an Associate Editor of the IEEE TRANSACTIONS ON SOFTWARE ENGINEERING.

Utilizing an Executable Specification Language for an Information System

SUSAN D. URBAN, JOSEPH E. URBAN, MEMBER, IEEE, AND WAYNE D. DOMINICK

Abstract—This paper describes an approach to software specification development with interpretation as applied to an information storage and retrieval system. Machine execution of software specifications is possible with both partial and complete specifications. A partial specification is interpreted using abstract execution. The Descartes specification language is utilized to describe a functional aspect of an existing information storage and retrieval system, namely, the MADAM (Multics Approach to Data Access and Management) system at the University of Southwestern Louisiana. Brief descriptions of both the Descartes language and the MADAM system precede the example specification. The paper concludes with a discussion of the expected results that this meth-

odology could have on the pragmatic development and evolution of information systems.

Index Terms—Abstract execution, executable specifications, information storage and retrieval, prototyping.

I. INTRODUCTION

Manuscript received June 1, 1984. The MADAM system was supported in part by the National Science Foundation and Northwestern University, Evanston, IL, under NSF Grant DSI76-19481; and portions of the continued evolution of the system were funded by the National Science Foundation under Grant SER77-06835.

The authors are with the Center for Advanced Computer Studies, University of Southwestern Louisiana, Lafayette, LA 70504.

IN 1968, the term "software engineering" was first used to suggest the need for a disciplined approach to the development of software. Eleven years later, Leon Stucki, Co-program Chairman of the Fourth International Conference on Software Engineering, noted: "Paradoxically, software engineering has done an excellent job of automating everyone else's job except its own" [1].

Software requirements for systems to access and manage large masses of data have motivated the use of general-purpose

computer-based information systems. An effective methodology for developing information systems is a concern of information analysts and software engineers. One step toward automating the job of the software engineer is the development of executable specification methodologies for translating system requirements into a working system model that can be understood by both computers and software engineers [2].

The field of software engineering is not the only area of computer science suffering from inadequate design and development methodologies. Many of the problems associated with the development of database management systems are similar to those of other types of software systems [3]. A current area of research is the application of software engineering methodologies to the development of information storage and retrieval systems and database management systems [4], [5].

The primary objective underlying this paper is a demonstration that an executable specification language can be successfully applied to the development of functional specifications for an information storage and retrieval system. The system addressed is the MADAM (Multics Approach to Data Access and Management) system at the University of Southwestern Louisiana, and the specification language employed is the Descartes language, also developed at the University of Southwestern Louisiana.

The Descartes specification language is a tool to be used throughout the software life cycle [6], [7]. After the formulation of the requirements of a system, Descartes is used to specify the functional relationship between the input and the output of the system. The requirements of the system can be checked against the specifications before design of the system. In the design phase, Descartes specifications provide precise information about what the system should be designed to do.

In addition to being a valuable tool in the initial development of a system, Descartes specifications are also valuable during the testing and maintenance phases. Test data can be input to both the specifications and the actual system. The output of the specifications and the output of the actual system can then be compared for consistency. During the maintenance phase, the specifications can be used as the primary source of information about the functional aspects of the system. Proposed changes can be worked out on the specifications before making any changes to the actual system.

Sections II and III of this paper will briefly overview the MADAM system and the Descartes language, respectively. Section IV will then present an example of the partial specification of a portion of the MADAM user language using Descartes. Finally, Section V will summarize the paper, identify the conclusions that can be drawn from this work, and highlight the major pragmatic implications that these research activities could have on the initial development and controlled evolution of both information systems and specification languages.

II. AN OVERVIEW OF THE MADAM INFORMATION STORAGE AND RETRIEVAL SYSTEM

The MADAM system is an interactive information storage and retrieval system developed within the DBMS project [8] of the Computer Science Department at the University of Southwestern Louisiana. MADAM was initially designed primarily

as a research tool to be used by faculty and graduate students while engaging in various aspects of research, design, implementation, analysis, and evaluation of information systems. The system has, however, developed into a useful tool for managing and accessing data within pragmatic, nonresearch types of data management applications.

MADAM is composed of the following primary functional subsystems.

1) *Database Definition Subsystem*: Facilities for defining the characteristics of a new database.

2) *Database Security Subsystem*: Facilities for defining or redefining the security controls to be enforced over a database.

3) *Database Maintenance Subsystem*: Facilities for maintaining a database, e.g., adding new entries, deleting existing entries, modifying existing entries.

4) *Database Inversion Subsystem*: Facilities for maintaining the inverted indexes into a database, e.g., adding and/or deleting inversion keywords or stopwords.

5) *Database Search Subsystem*: Facilities for searching a database, retrieving results, and displaying output.

6) *Database Redefinition Subsystem*: Facilities for redefining the characteristics of a database.

7) *System Administration Subsystem*: Facilities for managing, controlling, and administering the information system environment.

Tailored interfaces also exist linking MADAM to various textual, statistical, and graphical analysis packages, and a generalized interface exists allowing MADAM users to invoke any facility available within the Multics computing system.

The MADAM system has been instrumented with an extremely powerful and comprehensive software monitoring facility which is being used as the foundation for a major research and development effort addressing the measurement, analysis, and evaluation of information system execution efficiency and human-computer interactions within information system environments.

As the system continues to evolve in response to changing user needs, new application areas, and differing supportive computing environments, the importance of applying a repertoire of software tools to controlling this evolution is becoming more and more evident. The remainder of the paper will discuss the importance of one such software tool and will illustrate how such a language can be used effectively within an information system environment.

III. AN OVERVIEW OF THE DESCARTES SPECIFICATION LANGUAGE

An excellent review of existing specification methodologies is presented by Ramamoorthy and Ho in [9]. The intention of this section is to introduce the specification methodology provided by the Descartes specification language [10]. A brief overview of only the most salient features of the language is presented.

Descartes is a specification language which uses data structuring methods suggested by Hoare [11] to analyze program input into constituent parts that are then used to synthesize a description of program output. These data structuring methods are direct product, discriminated union, sequence, and recursion.

Jackson utilizes these data structuring methods to describe data in his methodology [12]. Hoare notes that these data structuring methods have analogous programming language counterparts in control structures.

A language processor has been developed for Descartes so that the specifications for a program can be executed on sample data for the program. An abstract execution feature allows each specification module to be incrementally developed on abstract and concrete data. Descartes specifications can be executed interactively [13] or in a single-step execution.

Descartes was evaluated with respect to criteria established by Liskov and Zilles [14]. These criteria include: formality, constructibility, comprehensibility, minimality, wide range of applicability, and extensibility. The research presented in this paper provides additional support for the evaluation of Descartes, especially in terms of constructibility, comprehensibility, applicability, and extensibility.

In Descartes, the approach to program specification is to first define the input and the output data. The output data are then related to the input data so that the output data becomes a function of the input data. This relationship is described in Descartes using "Hoare trees." Each Hoare tree structures its subtrees according to the data structuring methods mentioned above.

For example, direct product represents a concatenation of a set of elements. In programming language control structures, direct product is closely analogous to the compound statement. The Hoare tree:

```
select_command
  select
  ' '
  criteria
```

indicates that the node "select_command" has three subtrees, namely, "select", a blank space, and "criteria". The subtrees of a node are always indented directly below the node.

A discriminated union represents the selection of one element out of a set of elements. Discriminated union is closely analogous to the case construct in programming languages. The Hoare tree:

```
support_command+
  articles_only
  books_and_articles
  books_only
  display
  quit
```

for example, indicates that the node "support_command" can be one of the alternative subtrees listed below the node. The notation, "+", following a node name, always indicates a discriminated union.

A sequence indicates zero or more occurrences of a set of elements. In programming language control structures, a sequence is closely analogous to the construct of iteration. A sequence is indicated by following a node name with the symbol "*". Thus, the Hoare tree:

```
record*
  ASCII
```

```
COMMAND_TYPE_(A_COMMAND)
  A_COMMAND
    support_command+
      articles_only
      books_and_articles
      books_only
      display
      quit
    return
  SUPPORT_COMMAND+
    ARTICLES_ONLY
      'articles only'
    BOOKS_AND_ARTICLES
      'books and articles'
    BOOKS_ONLY
      'books only'
    DISPLAY
      'display'
    QUIT
      'quit'
```

Fig. 1. Sample specification module.

indicates that the node "record" is a sequence of zero or more occurrences of ASCII characters. When a specific range needs to be indicated for a sequence, the "*" may be replaced by a Pascal-like notation, such as "(1..50)", indicating that the node "record" must contain from 1 to 50 characters. The individual elements can be referenced as record(1), record(2), etc.

In a Descartes specification module, Hoare trees can either be analysis trees or synthesis trees. Analysis trees take a string of characters and describe it in terms of its constituent parts. The analysis of a string can be viewed as a pattern-matching process. The root node of an analysis tree becomes the name for the entire string, and each intermediate and terminal node becomes a name for the part of the string that it matches. Synthesis trees then produce the output of a specification by synthesizing previously analyzed strings together with literals and module calls.

Every node in a Hoare tree must either be a match node or a reference node. A match node (always written in lower-case letters) is a node that can acquire a value in one of two ways. One way is through the analysis of a string of characters. In an analysis tree, for example, a match node can acquire a value by matching the string being analyzed. Another way in which a match node can acquire a value is as the result of synthesis. Synthesized output is simply a string of characters whose value is given to a match node.

Any match node in a Hoare tree can be associated with a reference node of the same name. A reference node (always written in upper-case letters) is a node that returns the value acquired by its corresponding match node. In synthesis trees, reference nodes are used to build strings. Only a terminal reference node in a synthesis tree can actually return a value. An intermediate reference node, having a corresponding match node that has acquired a value, simply determines the subtree of the synthesis tree with which to continue the synthesis process.

Analysis trees always begin with a root node that is a reference node. The reference node indicates the string value that is to be analyzed. Synthesis trees always begin with a root node that is a match node. The match node gives a name to the synthesized value.

A simple example of a specification module is presented in Fig. 1 for the purpose of illustrating the use of the language

features discussed above. A module called `COMMAND_TYPE` takes a MADAM support command as an input string, analyzes the command to determine its type, and prints a message indicating the type of command recognized. The first tree in the module is an analysis tree. The value associated with the reference node `A_COMMAND` is analyzed to determine its command type. The nodes “support_command”, “articles_only”, “books_and_articles”, “books_only”, “display”, and “quit” are match nodes that can acquire a value as a result of analysis of `A_COMMAND`.

The second tree in the module is a synthesis tree that will return the message indicating the command type. The reference node `SUPPORT_COMMAND` acquires a value as a result of analyzing the input string. Furthermore, since the `SUPPORT_COMMAND` node is a discriminated union, one of the reference nodes listed below `SUPPORT_COMMAND` also acquires a value. The node that acquires a value is selected and the corresponding message is returned, indicating the command type.

Specifications written in Descartes are developed using a top-down approach. As a result, partial specifications can be tested before the refinement of lower level modules. Abstract execution provides a debugging feature for specification modules that have not been completely refined. A specification module that contains terminal match nodes is a candidate for abstract execution. For example, the specification module in Fig. 1 uses abstraction in the analysis of `A_COMMAND` since the detailed syntax of each command type has not yet been refined.

IV. AN EXAMPLE OF THE PARTIAL SPECIFICATION OF MADAM IN DESCARTES

The example presented in this section is a result of research reported in [15]. Emphasis is placed on the top-down development of the specifications. Testing of the specifications is illustrated through the use of abstract execution. The example has been extracted from a larger set of specifications and simplified for the purpose of this paper.

MADAM supports the storage and retrieval of bibliographic data. An important aspect in the application of a specification methodology to any information storage and retrieval system is the specification of the database. Since specifications written in Descartes are functional specifications, dealing with what is to be done rather than how it is to be done, the implementation details of the database are not important [16]. This implies that specifications must deal with the logical view of the data.

In Fig. 2, a Hoare tree is used to describe the logical view of a MADAM database. The node “records(1..)” indicates that the database consists of a sequence of one or more occurrences of “publication”. Each publication is a direct product, or an aggregate, of “record_number”, “author_list”, “affiliation_list”, “title”, “date_of_publication”, and “publication_type”. All items except “publication_type” represent the data items that are common to all publications within a MADAM database. Publications are therefore generalized with respect to these data items. The node “publication_type” indicates that publications can be either books or articles, each of which have

```

madam_database
  records(1.. )
    publication
      record_number
      author_list(1.. )
        author_name
      affiliation_list(1.. )
        affiliation
      title
      date_of_publication
      publication_type+
        book
          publisher
        article
          journal
          volume
          issue
          pages
          abstract
          keyword_list(1.. )
            keyword
          categories_list(1.. )
            category

```

Fig. 2. Hoare tree of a MADAM database.

additional data items that are unique to that particular publication type.

The portion of MADAM selected for specification within this section is a subset of the data base search subsystem. Before developing specifications for the search subsystem, a brief description of the subsystem is in order.

Database searching in MADAM is, by default, set to perform a free text search. While this is the most comprehensive mode of searching, it is often desired to restrict a search to only certain data items. This type of searching can be performed using the “select” command. In either case, the outcome of each valid search request is assigned a set number that can be referenced in subsequent commands.

The records of the database that are searched when using the “select” command or when performing a free text search are controlled by the setting of the search type. The search type is, by default, set to search for both articles and books. If a user wishes to restrict the search to only books, the “books only” command can be used. Likewise, a user can restrict the search to articles only by using the “articles only” command. The “books and articles” command allows a user to switch back to the default search type.

When using the “select” command, the user may enter the selection criteria in the form of a relational expression, a set expression, a free text expression, or any Boolean combination of the previous types of expressions. A subset of a sample interactive session between MADAM and a user using the “select” command is given below:

```

(MADAM) Please enter next term or system command
(MADAM) 1/
(user) select author eq "CODD";
(MADAM) 5 items
(MADAM) Please enter next term or system command
(MADAM) 2/
(user) select (title or keywords) equal ("DATA-
      BASE"
      or "RELATIONAL") and journal equal
      "CACM";
(MADAM) 10 items
(MADAM) Please enter next term or system command

```

(MADAM) 3/
 (user) select set 1 and set 2
 (MADAM) 3 items

In the above sample session, the first selection criterion is based on the relational expression "author eq CODD". The notation "1/" indicates the number that is assigned to the set of records satisfying the search. The second select command contains a Boolean combination of two relational expressions. The third select command performs a Boolean combination on the results of the first two database searches.

MADAM is also designed such that a user is provided with as precise a diagnostic message as possible when a syntax error occurs, with a user only having to reenter the portion of the "select" command starting from the point at which the error occurred. The example below illustrates the interaction between MADAM and a user when a syntax error occurs. MADAM prints a specific error message, along with a pointer to the location of the error within the research request and a message indicating the portion of the command to be reentered.

(MADAM) Please enter next term or system command
 (MADAM) 1/
 (user) select categories eq (3.7, 3.8, 3.9;
 (MADAM)
 (MADAM) A right parenthesis was expected.
 (MADAM) select categories eq (3.7, 3.8, 3.9;
 (MADAM)|
 (MADAM) Please retype the following portion of your input.
 (MADAM) ;
 (user));

Due to the complexity of the "select" command, precise specification of the functional relationship between a search request and the resulting MADAM output is important to the design, development, and maintenance of an analyzer and interpreter for the command. The syntax of the "select" command is currently specified using a BNF definition. Semantics of the command are stated using an English language description. Other than a list of diagnostic messages, there are no specifications stating the conditions that initiate the use of each diagnostic message. Descartes can be used to combine the specifications for the syntax and semantics of the "select" command to establish a relationship between user input and MADAM output.

Given the above description for the database search subsystem, the specifications can now be developed and tested in a top-down fashion using abstract execution. For the purpose of this paper, the specifications will focus on the use of the select command to perform a search based on a single relational expression. Boolean combinations of expressions will not be addressed.

Fig. 3 illustrates the top-most module in the specification of the database search subsystem. This specification module is invoked from within another specification module which controls access to the database search subsystem. The calling specification module passes four input parameters to the

```

SEARCH_SUBSYSTEM_(DATABASE)_(SEARCH_TYPE)_(HIT_SETS)_(USER)
  DATABASE
    records*
      ASCII
  SEARCH_TYPE
    type*
      ASCII
  HIT_SETS
    sets*
      previous_search_set*
        record_number
          INTEGER
        'xxx'
  USER
    directive+
      support_command+
        articles_only
        books_and_articles
        books_only
        display
        quit
      select_command
        select
          ' '
        criteria
        free_text_operation
        free_text
  NL
  leftover*
    ASCII
return
  'Please input next term or system command'
  NL
  (SETS#)_PLUS_(1)
  '/'
  NL
  DIRECTIVE+
    SUPPORT_COMMAND+
      ARTICLES_ONLY
        'articles only executed'
        SEARCH_SUBSYSTEM_(RECORDS)_(TYPE)_(SETS)_(LEFTOVER)
      BOOKS_AND_ARTICLES
        'books and articles executed'
        SEARCH_SUBSYSTEM_(RECORDS)_(TYPE)_(SETS)_(LEFTOVER)
      BOOKS_ONLY
        'books only executed'
        SEARCH_SUBSYSTEM_(RECORDS)_(TYPE)_(SETS)_(LEFTOVER)
      DISPLAY
        'display executed'
        SEARCH_SUBSYSTEM_(RECORDS)_(TYPE)_(SETS)_(LEFTOVER)
      QUIT
        'quit executed'
    SELECT_COMMAND
        'select executed'
        SEARCH_SUBSYSTEM_(RECORDS)_(TYPE)_(SETS)_(LEFTOVER)
    FREE_TEXT_OPERATION
        'free text search executed'
        SEARCH_SUBSYSTEM_(RECORDS)_(TYPE)_(SETS)_(LEFTOVER)

```

Fig. 3. Top module in specification of MADAM database search subsystem.

module in Fig. 3: DATABASE, USER, SEARCH_TYPE, and HIT_SETS.

The DATABASE string contains the database records that are to be searched. The structure of the records is described by the Hoare tree in Fig. 2.

The USER string contains the string of user commands that could be entered during a MADAM session. USER can be described by the following Hoare tree:

```

user
  input_from_terminal(1 .. )
  command_line*
  ASCII
  NL

```

The above Hoare tree does not attempt to relate input to output. It is an example of using a Hoare tree to describe the nature of the input data. In particular, the Hoare tree above describes input from the user as being a sequence of one or more occurrences of a command line, with each command line followed by a new-line character (NL). Each command line is a sequence of zero or more occurrences of an ASCII character. The nodes ASCII and NL are module calls to Hoare trees that

refine the description of an ASCII character and a new-line character, respectively.

The `SEARCH_TYPE` string contains a string that indicates whether the database search is to be restricted to books only, books and articles, or articles only. When the module in Fig. 3 is initially invoked, `SEARCH_TYPE` contains the string "books and articles" (the system default setting).

The `HIT_SETS` string contains the sequence of the list of record numbers indentifying the records satisfying previous searches. `HIT_SETS` is described by the following Hoare tree:

```
HIT_SETS
sets*
  previous_search_set*
    record_number
      INTEGER
    ,
  'xxx'
```

The nodes "sets" is a sequence of zero or more occurrences of "previous_search_set", followed by the string "xxx". The node "previous_search_set" is a sequence of zero or more occurrences of "record_number", followed by a blank. The node `INTEGER`, such as `ASCII` and `NL`, is a module call to a specification module that refines the description of an integer. The blank is inserted to separate record numbers and the string "xxx" is inserted to indicate the end of each search set. When the module in Fig. 3 is initially called, the `HIT_SETS` input argument is a null string.

The first two analysis trees in the module in Fig. 3 simply analyze `DATABASE` and `SEARCH_TYPE` to be strings of ASCII characters. The match nodes, "records" and "type", acquiring values as a result of the analysis, will be passed to other modules that perform the actual database search. Strings can therefore be analyzed in different ways by different modules. In this particular module, there is no need to perform a detailed analysis of `DATABASE` and `SEARCH_TYPE`. The refinement of this example will eventually yield a module in which the detailed analysis of `DATABASE` and `SEARCH_TYPE` is required.

The next analysis tree in the specification module in Fig. 3 analyzes `HIT_SETS` as a means of counting the number of previous searches that have occurred. (Note that `HIT_SETS` also provides information about the specific record numbers satisfying each previous search. The module in Fig. 3, however, only needs information about the number of previous searches that have occurred.) The count is used to assign a number to the next search performed. Since `HIT_SETS` is initially null, the count will be zero before performing the first search. The count is extracted from the analyzed string by following the reference node, `SETS`, with a sharp sign ("#"). The number to be assigned to the hit set will be calculated by invoking the Descartes `PLUS` module to add one to the count. Use of the sharp sign is illustrated in the third line of the "return" synthesis tree in Fig. 3. The message printed by the first five lines of the synthesis tree when the count is zero will appear as:

```
Please enter next term or system command
1/
```

The `USER` string in Fig. 3 is analyzed to determine the search subsystem directive to perform. The search directive can either be a support command, the select command, or input for a free text search. If the directive is the select command, the command is followed immediately by the search criteria. The remaining characters in `USER` are collected into the node "leftover."

The last tree in Fig. 3 is a synthesis tree which uses the analyzed input string to output the prompt for the search directive. It also determines the search directive to perform through the `DIRECTIVE` reference node. If the user input matched the "support_command" node in the `USER` analysis tree, then the synthesis subtree alternative beginning with the reference node `SUPPORT_COMMAND` is selected. The reference node for this support command that was matched will then be selected to return a value. For example, if the "articles_only" match node acquired a value during the analysis of `USER`, the `ARTICLES_ONLY` reference node in the synthesis tree will return the string "articles only executed," and then recursively call the module in Fig. 3 to determine the next search directive. If the user input matched the "select_command" node or the "free_text" node during the analysis of `USER`, then the corresponding synthesis subtree alternative is selected.

The specification developed can be tested using abstract execution. In the analysis of the `USER` string, no syntax is specified for the support commands, the select command, or the input of a free text search. In all cases, terminal match nodes are used instead. As stated in the previous section, terminal match nodes are utilized in abstract execution. The language processor for Descartes recognizes that a terminal match node indicates a subtree that has not been completely refined. The name of the match node can be used in the input string as a means of testing the specification.

As an example, suppose the `USER` string consists of the following search directive:

```
select criteria
articles_only
free_text
display
quit
```

The output of the specification module in Fig. 3 is the following:

```
Please enter next term or system command
1/
select executed
Please enter next term or system command
1/
articles only executed
Please enter next term or system command
1/
free text search executed
Please enter next term or system command
1/
display executed
Please enter next term or system command
1/
quit executed
```

The output consists of the prompts for the search directives and messages indicating that each search directive from the USER string is recognized. The search set number would normally be incremented by one after the execution of the select command or a free text search. The number remains constant in the above output since the specification has not been refined to describe a search of the database.

After testing the specification module in Fig. 3, each command can be refined and tested. Refinement of the "select" command node, i.e., the terminal match nodes under the "select" command node in the USER analysis tree, is changed to reflect the actual syntax of the command. The command can either be input as the word "select" or the abbreviation "se." The terminal match node for the "select" command is refined as follows:

```
select_command+
  select+
    'select '
    'se '
  criteria*
  ASCII
```

The reference node for the select command in the "return" synthesis subtree is then refined to invoke the specification module that begins the analysis of the search criteria:

```
SELECT
  SEARCH_KEY_(RECORDS)_ (TYPE)_ (SETS)_
  (LEFTOVER)_ (CRITERIA)
```

The arguments passed to the invoked module are the reference node names for portions of the analyzed arguments in Fig. 3, e.g., RECORDS is the name of the analyzed DATABASE argument and LEFTOVER is the name of the remaining portion of the analyzed USER argument.

Within the SEARCH_KEY module, the exact syntax of CRITERIA can then be refined, again using abstraction in the incremental development of the specifications. The Hoare tree in Fig. 4 provides an abstract description of the search criteria for the "select" command. According to Fig. 4, the following are valid abstract requests.

```
not expression and expression
(expression or expression) and _not expression
expression and (expression or _not (expression or
expression))
```

The abstract view of search criteria implies that there are two levels of syntax analysis that must be performed. The first level involves analyzing the syntax of the Boolean combination of expressions. The second level involves analyzing the syntax of each individual expression. Using the abstract execution feature of Descartes, it is possible to develop and test the syntax analysis specifications in a top-down fashion, complete with specifications for error-handling facilities. Refinement of the second level specifications can be postponed until specification of the first level is complete.

This concludes the example specification provided in this paper. Further discussion of the complete specification is

```
search request
  leading_operator+
  'not'
  ''
  request+
    expression
    parenthesized_expression
    '('
    search_request
    ')'
    boolean_combination
    search_request
    boolean_operator+
    and_operator+
    and
    and_not
    nand
    or_operator+
    or
    or_not
    nor
    search_request
    parenthesized_boolean_combination
    '('
    search_request
    boolean_operator+
    and_operator+
    and
    and_not
    nand
    or_operator
    or
    or_not
    nor
    search_request
    ')'
  ''
```

Fig. 4. Hoare tree of search criteria.

provided in [14]. The development of the refined modules, such as "SEARCH_KEY_...", proceeds in a manner as described above.

V. CONCLUSION

This paper demonstrates the application of a specification methodology to an information storage and retrieval system. The methodology presented is an executable specification language that supports and encourages the top-down development of specifications. An abstract execution feature is available to support the testing of partial specifications.

Although MADAM is currently an implemented system, the development of executable specifications for the system is advantageous for future system revisions. As changes to the system are proposed, the changes can be initially reflected through the specifications. The proposed changes can then be refined, working strictly with the system model that the specifications represent. Once the proposed changes are developed on the system model, the changes can be implemented in the actual system. Results of testing the actual system can then be validated against the results of the system model.

MADAM is the largest system to which Descartes has been applied. As mentioned in Section III, the results of this research provide additional support for the evaluation of Descartes in terms of constructibility, comprehensibility, applicability, and extensibility. The constructibility of the language, or the ease with which the specifications can be developed, can be readily observed in the initial subsystem specification module in Fig. 2. This module is sufficient to give a global view of the types of facilities supported by the subsystem. Through the use of abstract execution, the module was tested before the refinement of the specifications of each facility. The modular approach to the development of the specifications

allows the developer to concentrate on manageable portions of the global concept. Comprehensibility, or the degree of difficulty involved in the reconstruction of the concept the specifications represent, is determined by those who use the specifications to make future revisions to MADAM.

The applicability of Descartes to an information storage and retrieval system is evident by the example presented in this paper. MADAM does, however, have a simple database structure. Therefore, the degree of applicability of Descartes to information storage and retrieval systems in general needs to be determined by applying Descartes to systems with more complex database structures.

The final evaluation criteria to be discussed is extensibility, or the degree of change required within the specifications due to a minimal change in the requirements. The specifications developed in this paper are an extension of an earlier version of MADAM in which three separate prompts were provided for the select command, the data item to be searched, and the query. The actual specifications allow a user to take either approach when using the select command. The specifications have been modified to recognize that the select command can either be entered on a line by itself (indicating a desire to be prompted for the search criteria), or on a line followed immediately by the search criteria (with no additional prompting).

In conclusion, we believe this research reinforces the importance of executable specifications to the software life cycle. Design errors typically account for a large portion of the total software errors of a system. A specification language such as Descartes can potentially reduce the cost of software development by placing greater emphasis on the initial specification and design phases of a system. Complete and precise specifications can also lead to the development of more reliable software.

REFERENCES

- [1] *Proc. 4th Int. Conf. Software Eng.*, Munich, Germany, Sept. 17-19, 1979.
- [2] C. V. Ramamoorthy, S. F. Ho, and H. H. So, "The role of software tools in a methodology for the development and validation of critical software for nuclear power plants," in *Tutorial: Software Methodology*, C. V. Ramamoorthy and R. T. Yeh, Eds., publ. by IEEE, New York, Cat. No. EHO 142-0, Nov. 1978, pp. 397-415.
- [3] A. I. Wasserman, "A software engineering view of data base management," in *Proc. 4th Int. Conf. Very Large Data Bases*, West Berlin, Germany, Sept. 13-15, pp. 23-35.
- [4] A. I. Wasserman and S. K. Stinson, "A specification method for interactive information systems," in *Proc. Specification of Reliable Software Conf.*, Cambridge, MA, Apr. 3-5, 1979, pp. 68-79.
- [5] R. T. Yeh and J. W. Baker, "Toward a design methodology for DBMS: Software engineering approach," in *Proc. 3rd Int. Conf. Very Large Data Bases*, Tokyo, Japan, Oct. 1977.
- [6] J. E. Urban, "Software development with executable functional specifications," in *Proc. 6th Int. Conf. Software Eng.*, Tokyo, Japan, Sept. 13-16, pp. 418-419.
- [7] —, "Abstract execution in the Descartes specification language," *IEEE Trans. Software Eng.*, to be published.
- [8] W. D. Dominick, "The USL DBMS Project," Dep. Comput. Sci., Univ. Southwestern Louisiana, Lafayette, Tech. Rep. CMPS-78-6-5, Sept. 1978.
- [9] C. V. Ramamoorthy and H. H. So, "Software requirements and specifications: Status and perspectives," in *Tutorial: Software Methodology*, C. V. Ramamoorthy and R. T. Yeh, Eds., Catalog Publ. by IEEE, New York, Catalog No. EHO 142-0, Nov. 1978, pp. 43-164.
- [10] J. E. Urban, "A specification language and its processor," Ph.D. dissertation, Dep. Comput. Sci., Univ. Southwestern Louisiana, Lafayette, Dec. 1977.

- [11] C. A. R. Hoare, "Data reliability," *SIGPLAN Notices*, vol. 10, pp. 528-533, June 1975.
- [12] M. A. Jackson, *Principles of Program Design*. New York: Academic, 1976.
- [13] S. K. Tang, "Enhancements on the language processor for the Descartes specification language," M.S. Project Rep., Dep. Comput. Sci., Univ. Southwestern Louisiana, Lafayette, Dec. 1981.
- [14] B. H. Liskov and S. Zilles, "Specification techniques for data abstractions," *SIGPLAN Notices*, vol. 10, pp. 72-87, June 1975.
- [15] S. D. Urban, "The application of a specification methodology to an information storage and retrieval system, M. S. thesis, Dep. Comput. Sci., Univ. Southwestern Louisiana, Lafayette, July 1980.
- [16] R. Balzer and N. Goldman, "Principles of good software specification and their implications for specification languages," in *Proc. Specification Reliable Software Conf.*, Cambridge, MA, Apr. 3-5, 1979, pp. 58-67.

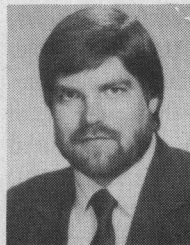


Susan D. Urban received the B.S. and the M.S. degrees in computer science in 1976 and 1980, respectively, from the University of Southwestern Louisiana, Lafayette, where she is now pursuing the Ph.D. degree in computer science.

She is currently a part-time instructor at the Center for Advanced Computer Studies of the University of Southwestern Louisiana. She has worked as an Information Analyst at the Medical College of Georgia, Augusta, and as a Programmer/Analyst at Southern Flow Companies

in Lafayette, LA. Her research interests include software engineering, database management systems, and knowledge-based systems.

Ms. Urban is a member of the Association for Computing Machinery and the IEEE Computer Society.



Joseph E. Urban (M'82) received the B.S. degree from Florida Institute of Technology, Melbourne, the M.S. degree from the University of Iowa, Iowa City, and the Ph.D. degree from the University of Southwestern Louisiana, Lafayette, in 1973, 1975, and 1977, respectively, all in computer science.

He is now an Associate Professor of Computer Science at the University of Southwestern Louisiana, responsible for the software engineering program. He has worked at the U.S.

Army Signal Center and the University of South Carolina. His research interests include specification languages, programming languages, software engineering, and computer science education.

Dr. Urban was the founder and initial Chairman of the IEEE Computer Society Computer Languages Technical Committee. He is currently a member of the IEEE Computer Society Governing Board and an Associate Editor of IEEE TRANSACTIONS ON SOFTWARE ENGINEERING. He is a member of the Association for Computing Machinery and the IEEE Computer Society.



Wayne D. Dominick received the Ph.D. degree in computer science from Northwestern University, Evanston, IL, in 1975.

He is currently an Associate Professor with the Center for Advanced Computer Studies of the University of Southwestern Louisiana, Lafayette, and an officer and member of the Board of Directors of two computer science/information systems consulting corporations. His research and development interests include the planning and administration of information system environments, information system (DBMS, IS&R, MIS) design, system performance measurement and evaluation, and user/system interface design/evaluation. He has over 90 publications and presentations and has consulted widely throughout the United States, Canada, Europe, and Japan.

Reproduced with permission of the copyright owner. Further reproduction prohibited without permission.